Compte Rendu – TP 4

Architecture Orientée Service (SOA)

Année universitaire : 2025 / 2026

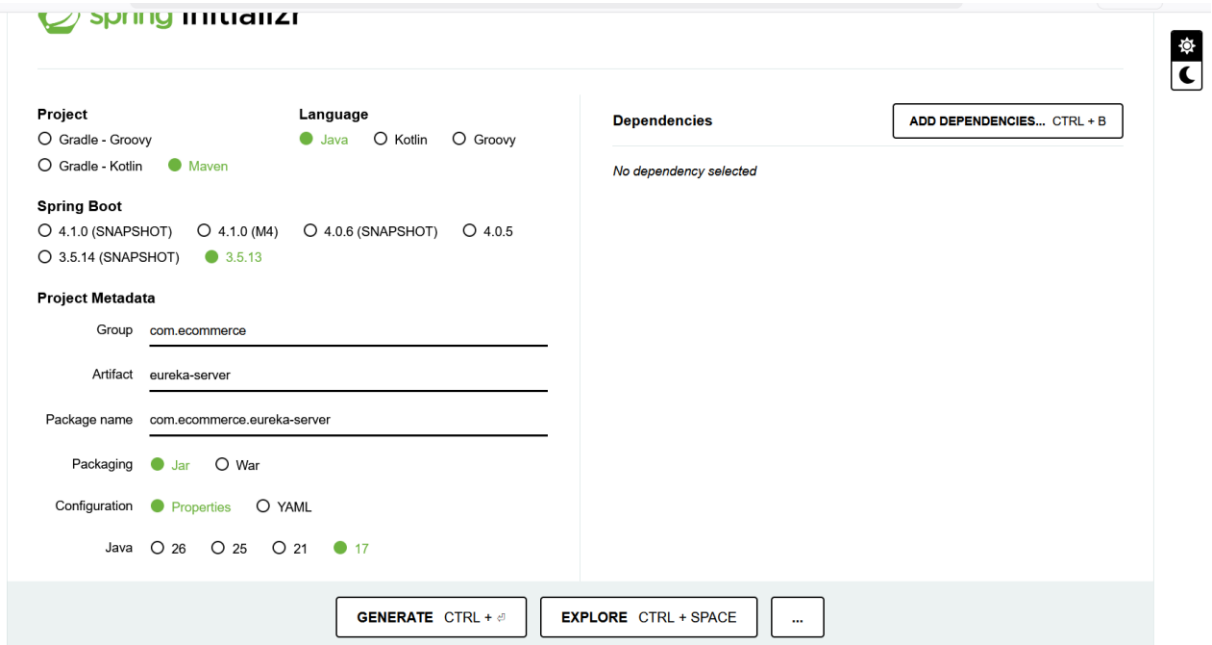
Réalisé par : SALMA ASSAMRI

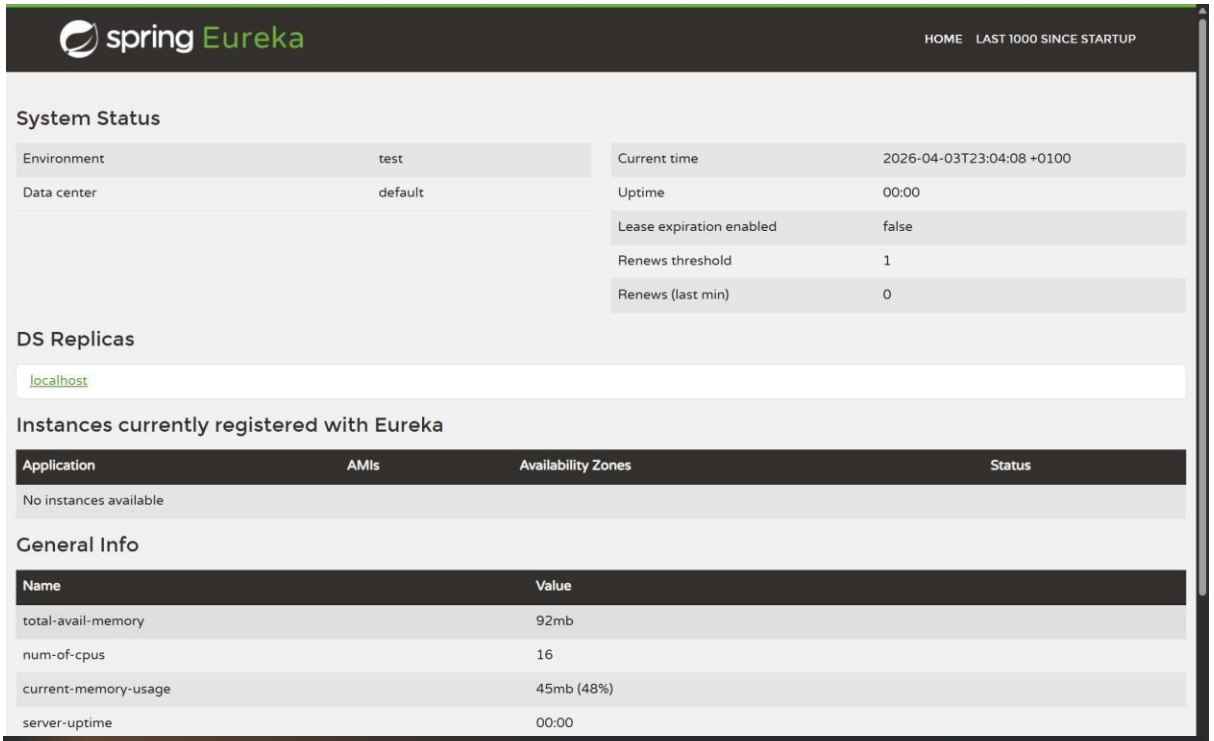
Encadré par : Pr. BE ELBAGHAZAOU

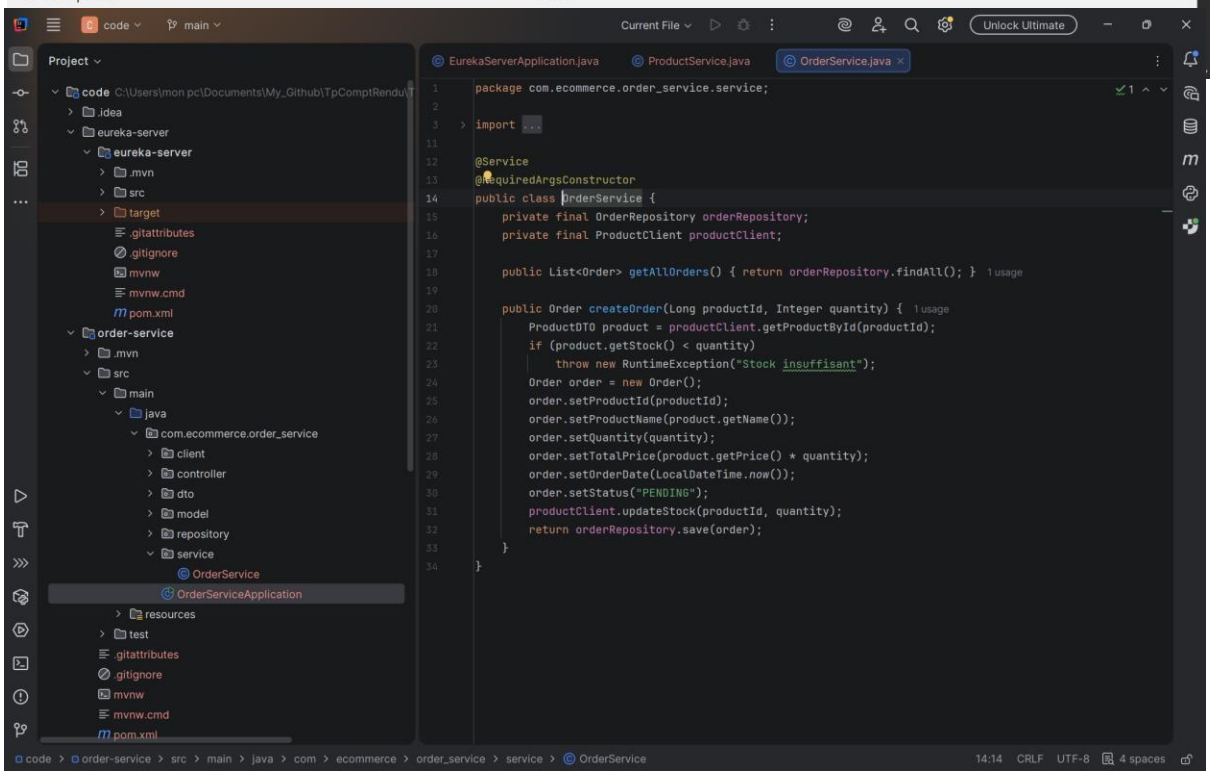
Objectifs :

- Assimiler les bases de l'architecture orientée services (SOA).
- Développer des microservices avec Spring Boot.
- Mettre en place la communication entre services via REST.
- Manipuler les données avec Spring Data JPA.
- Enregistrer et découvrir les services grâce à Eureka.

Etape 1 :




spring Eureka
HOME
LAST 1000 SINCE STARTUP
System Status
Environment
test
Data center
default
Current time
2026-04-03T23:04:08 +0100
Uptime
00:00
Lease expiration enabled
false
Renews threshold
1
Renews (last min)
0
DS Replicas
localhost
Instances currently registered with Eureka
Application
AMIs
Availability Zones
Status
No instances available
General Info
Name
Value
total-avail-memory
92mb
num-of-cpus
16
current-memory-usage
45mb (48%)
server-uptime
00:00


code
main
Current File
Unlock Ultimate
Project
EurekaServerApplication.java
ProductService.java
OrderService.java
package com.ecommerce.order_service.service;
import ...
@Service
@RequiredArgsConstructor
public class OrderService {
private final OrderRepository orderRepository;
private final ProductClient productClient;
public List<Order> getAllOrders() { return orderRepository.findAll(); }
public Order createOrder(Long productId, Integer quantity) {
ProductDTO product = productClient.getProductById(productId);
if (product.getStock() < quantity)
throw new RuntimeException("Stock insuffisant");
Order order = new Order();
order.setProductId(productId);
order.setProductName(product.getName());
order.setQuantity(quantity);
order.setTotalPrice(product.getPrice() * quantity);
order.setOrderDate(LocalDateTime.now());
order.setStatus("PENDING");
productClient.updateStock(productId, quantity);
return orderRepository.save(order);
}
}
code > order-service > src > main > java > com > ecommerce > order_service > service > OrderService
14:14
CRLF
UTF-8
4 spaces

Etape 2 :

spring initializr

Project

- Gradle - Groovy
- Gradle - Kotlin
- Maven

Language

- Java
- Kotlin
- Groovy

Spring Boot

- 4.1.0 (SNAPSHOT)
- 4.1.0 (M4)
- 4.0.6 (SNAPSHOT)
- 4.0.5
- 3.5.14 (SNAPSHOT)
- 3.5.13

Project Metadata

Group: com.ecommerce

Artifact: product-service

Package name: com.ecommerce.product-service

Packaging: Jar / War

Configuration: Properties / YAML

Java: 26 / 25 / 21 / 17

Dependencies — ADD DEPENDENCIES... CTRL + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

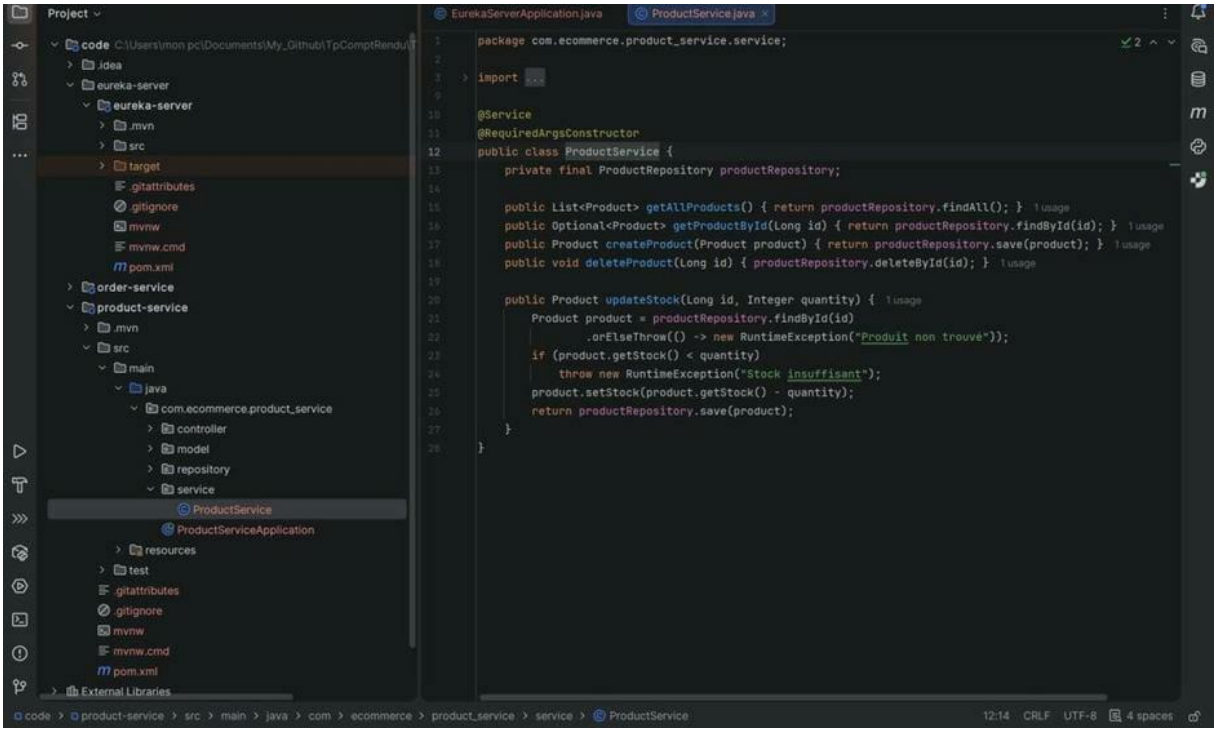
Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

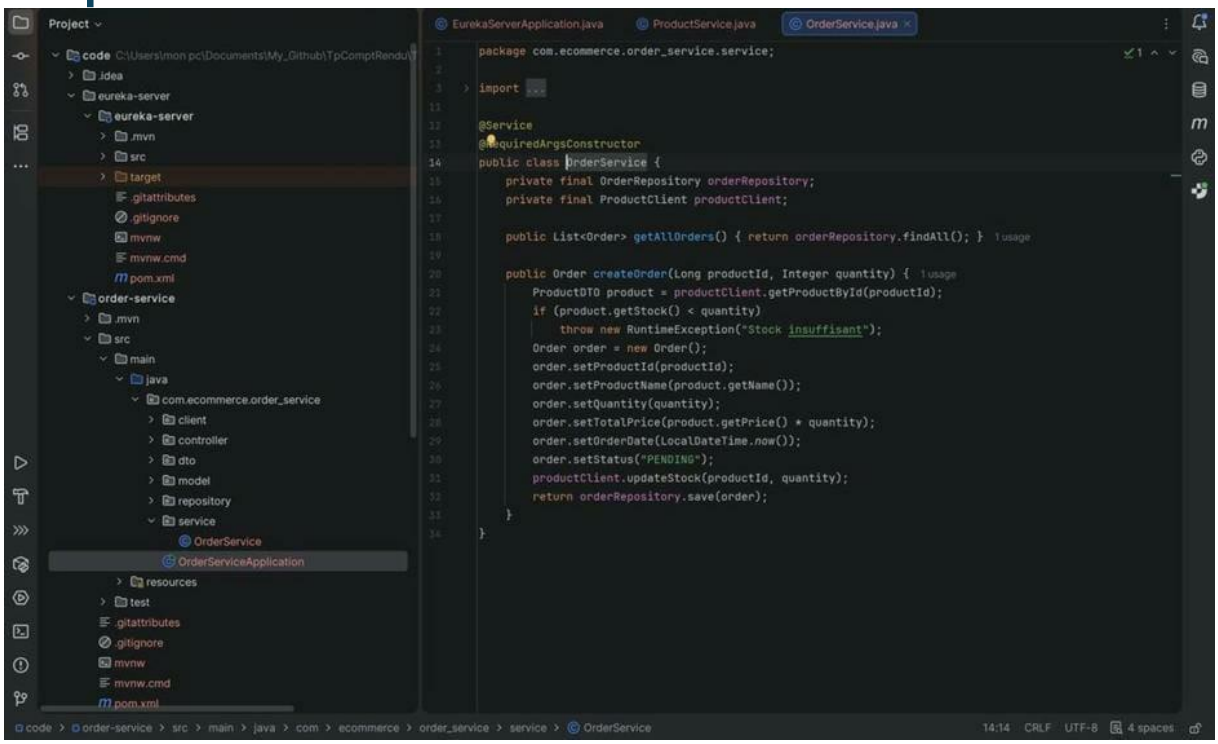
Eureka Discovery Client SPRING CLOUD DISCOVERY
A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

Lombok DEVELOPER TOOLS
Java annotation library which helps to reduce boilerplate code.

GENERATE CTRL + ⏎ | EXPLORE CTRL + SPACE | ...

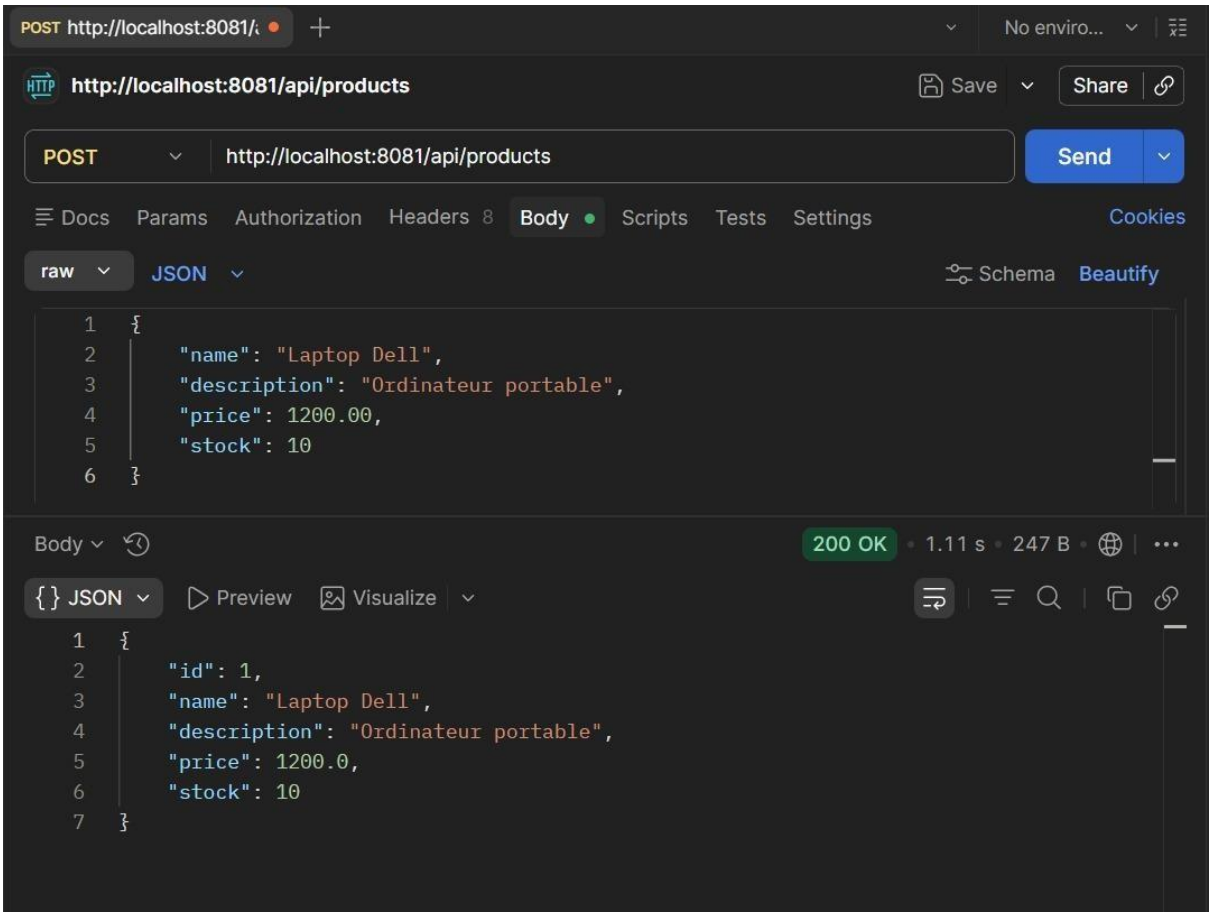

Project
code C:\Users\mon pc\Documents\My_Github\TpComptRendu\T
.idea
eureka-server
eureka-server
.mvn
src
target
.gitattributes
.gitignore
mvnw
mvnw.cmd
pom.xml
order-service
product-service
.mvn
src
main
java
com.ecommerce.product_service
controller
model
repository
service
ProductService
ProductServiceApplication
resources
test
.gitattributes
.gitignore
mvnw
mvnw.cmd
pom.xml
External Libraries
EurekaServerApplication.java
ProductService.java
package com.ecommerce.product_service.service;
import ...
@Service
@RequiredArgsConstructor
public class ProductService {
private final ProductRepository productRepository;
public List<Product> getAllProducts() { return productRepository.findAll(); } 1 usage
public Optional<Product> getProductById(Long id) { return productRepository.findById(id); } 1 usage
public Product createProduct(Product product) { return productRepository.save(product); } 1 usage
public void deleteProduct(Long id) { productRepository.deleteById(id); } 1 usage
public Product updateStock(Long id, Integer quantity) { 1 usage
Product product = productRepository.findById(id)
.orElseThrow(() -> new RuntimeException("Produit non trouvé"));
if (product.getStock() < quantity)
throw new RuntimeException("Stock insuffisant");
product.setStock(product.getStock() - quantity);
return productRepository.save(product);
}
}
code > product-service > src > main > java > com > ecommerce > product_service > service > ProductService
12:14 CRLF UTF-8 4 spaces

Etape 3 :

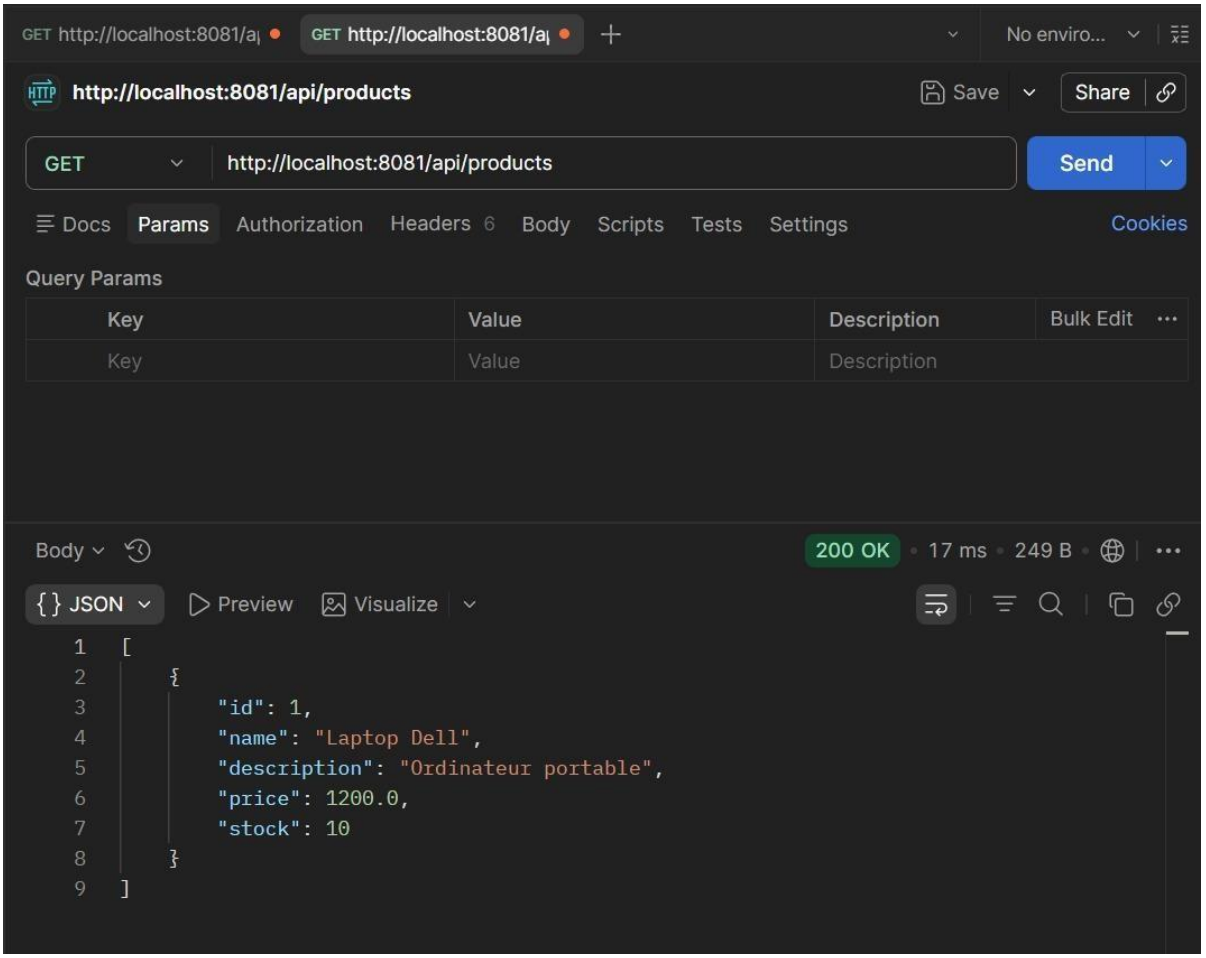

Project
code C:\Users\mon pc\Documents\My_Github\TpComptRendu\T
.idea
eureka-server
eureka-server
.mvn
src
target
.gitattributes
.gitignore
mvnw
mvnw.cmd
pom.xml
order-service
.mvn
src
main
java
com.ecommerce.order_service
client
controller
dto
model
repository
service
OrderService
OrderServiceApplication
resources
test
.gitattributes
.gitignore
mvnw
mvnw.cmd
pom.xml
EurekaServerApplication.java
ProductService.java
OrderService.java
package com.ecommerce.order_service.service;
import ...
@Service
@RequiredArgsConstructor
public class OrderService {
private final OrderRepository orderRepository;
private final ProductClient productClient;
public List<Order> getAllOrders() { return orderRepository.findAll(); } 1 usage
public Order createOrder(Long productId, Integer quantity) { 1 usage
ProductDTO product = productClient.getProductById(productId);
if (product.getStock() < quantity)
throw new RuntimeException("Stock insuffisant");
Order order = new Order();
order.setProductId(productId);
order.setProductName(product.getName());
order.setQuantity(quantity);
order.setTotalPrice(product.getPrice() * quantity);
order.setOrderDate(LocalDateTime.now());
order.setStatus("PENDING");
productClient.updateStock(productId, quantity);
return orderRepository.save(order);
}
}
code > order-service > src > main > java > com > ecommerce > order_service > service > OrderService
14:14 CRLF UTF-8 4 spaces

Tests :

Test 1 :



Test 2 :


GET http://localhost:8081/a
GET http://localhost:8081/a
No enviro...
http://localhost:8081/api/products
Save
Share
GET
http://localhost:8081/api/products
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
200 OK
17 ms
249 B
JSON
Preview
Visualize
[
{
"id": 1,
"name": "Laptop Dell",
"description": "Ordinateur portable",
"price": 1200.0,
"stock": 10
}
]

Test 3 :

GET http://localhost:8081/a | GET http://localhost:8081/a | POST http://localhost:8082/ | No enviro...

http://localhost:8082/api/orders

Save | Share

POST http://localhost:8082/api/orders?productId=1&quantity=2 Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

	Key	Value	Description	Bulk Edit
✓	productId	1		
✓	quantity	2		
	Key	Value	Description	

Body | 200 OK | 1.54 s | 301 B

JSON | Preview | Visualize


```
1 {
2     "id": 1,
3     "productId": 1,
4     "productName": "Laptop Dell",
5     "quantity": 2,
6     "totalPrice": 2400.0,
7     "orderDate": "2026-03-26T21:41:58.5924606",
8     "status": "PENDING"
9 }
```


Test 4 :

GET http://localhost | GET http://localhost | POST http://localhos | GET http://localhost | No enviro...

http://localhost:8082/api/orders

Save | Share

GET http://localhost:8082/api/orders Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 567 ms • 302 B

{ } JSON | Preview | Visualize


```
[
    {
        "id": 1,
        "productId": 1,
        "productName": "Laptop Dell",
        "quantity": 2,
        "totalPrice": 2400.0,
        "orderDate": "2026-03-26T21:41:58.592461",
        "status": "PENDING"
    }
]
```